

MLDL Experiment 9

Aim: To Implement Recurrent Neural Network (RNN) / LSTM for time series or text data.

Theory:

1. Dataset Source

Source: [International Airline Passengers \(Kaggle\)](#)

2. Dataset Description

- **Nature:** Univariate Time Series.
- **Size:** 144 monthly observations (from January 1949 to December 1960).
- **Features:** **Month** (Time index) and **Passengers** (Total count in thousands).
- **Target Variable:** The number of passengers in the next month ($t+1$).
- **Characteristics:** Shows both a clear **upward trend** and **seasonal patterns** (peaks during vacation months).

3. Mathematical Formulation of LSTM

Recurrent Neural Networks (RNNs) suffer from the "vanishing gradient" problem. **Long Short-Term Memory (LSTM)** units solve this using "gates" that control the flow of information.

The key equations for an LSTM cell at time t are:

1. **Forget Gate (f_t):** Decides what to discard from the previous state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

2. **Input Gate (i_t):** Decides which new information to store.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

3. **Cell State Update (C_t):** Updates the internal memory.

$$C_t = f_t * C_{t-1} + i_t * \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

4. **Output Gate (o_t):** Decides what to output as the hidden state (h_t).

$$h_t = o_t * \tanh(C_t)$$

4. Algorithm Limitations

- **Data Requirement:** LSTMs are complex and can easily overfit on very small datasets like this one if not regularized.
- **Sensitivity to Scaling:** LSTMs are highly sensitive to the scale of input data (e.g., values 100 vs 500); **Min-Max Scaling** is mandatory.
- **Stationarity:** They perform better if trends are removed, though deep LSTMs can often learn trends directly.

5. Methodology / Workflow

1. **Data Preprocessing:** Convert the data into a float type and normalize to a $[0, 1]$ range.
2. **Windowing:** Transform the series into a supervised learning format. For example, use a **look_back** of 12 months (one full year) to predict the 13th month.
3. **Reshaping:** Reshape the input into $[\text{samples}, \text{time_steps}, \text{features}]$.
4. **Model Building:** A Sequential model with one LSTM layer and one Dense output layer.
5. **Training:** Minimize Mean Squared Error (MSE) over 100 epochs.
6. **Prediction:** Forecast on the test set and "inverse transform" the values back to the original passenger counts.

6. Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error

# 1. Load Dataset
url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv'
df = pd.read_csv(url, usecols=[1])
dataset = df.values.astype('float32')

# 2. Preprocess: Normalize data to [0, 1]
scaler = MinMaxScaler(feature_range=(0, 1))
dataset_scaled = scaler.fit_transform(dataset)

# 3. Create supervised learning structure
def create_dataset(data, look_back=1):
```

```

X, Y = [], []
for i in range(len(data)-look_back-1):
    X.append(data[i:(i+look_back), 0])
    Y.append(data[i + look_back, 0])
return np.array(X), np.array(Y)
look_back = 12
X, y = create_dataset(dataset_scaled, look_back)

# Split into train and test (80/20 split)
train_size = int(len(X) * 0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:] # FIXED: Changed y_size to train_size

# Reshape input to be [samples, time steps, features]
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

# 4. Build LSTM Model
model = Sequential([
    LSTM(50, activation='relu', input_shape=(look_back, 1)),
    Dense(1)
])
model.compile(optimizer='adam', loss='mse')

# 5. Train
print("Training LSTM... this may take a moment.")
model.fit(X_train, y_train, epochs=100, batch_size=1, verbose=0)

# 6. Predict
train_predict = model.predict(X_train)
test_predict = model.predict(X_test)

# Invert predictions back to original scale
train_predict = scaler.inverse_transform(train_predict)
test_predict = scaler.inverse_transform(test_predict)
y_test_actual = scaler.inverse_transform(y_test.reshape(-1, 1))

```

7. Plotting

```
plt.figure(figsize=(12,6))
```

```
plt.plot(dataset, label='Actual Monthly Passengers', color='blue')
```

```
# Correctly align the test predictions on the plot
```

```
# The test plot starts after the training data + look_back window
```

```
test_plot_start = len(train_predict) + look_back
```

```
test_plot_indices = np.arange(test_plot_start, test_plot_start + len(test_predict))
```

```
plt.plot(test_plot_indices, test_predict, label='LSTM Prediction', color='red')
```

```
plt.title('International Airline Passengers Forecast')
```

```
plt.xlabel('Months (from Jan 1949)')
```

```
plt.ylabel('Passengers (1000s)')
```

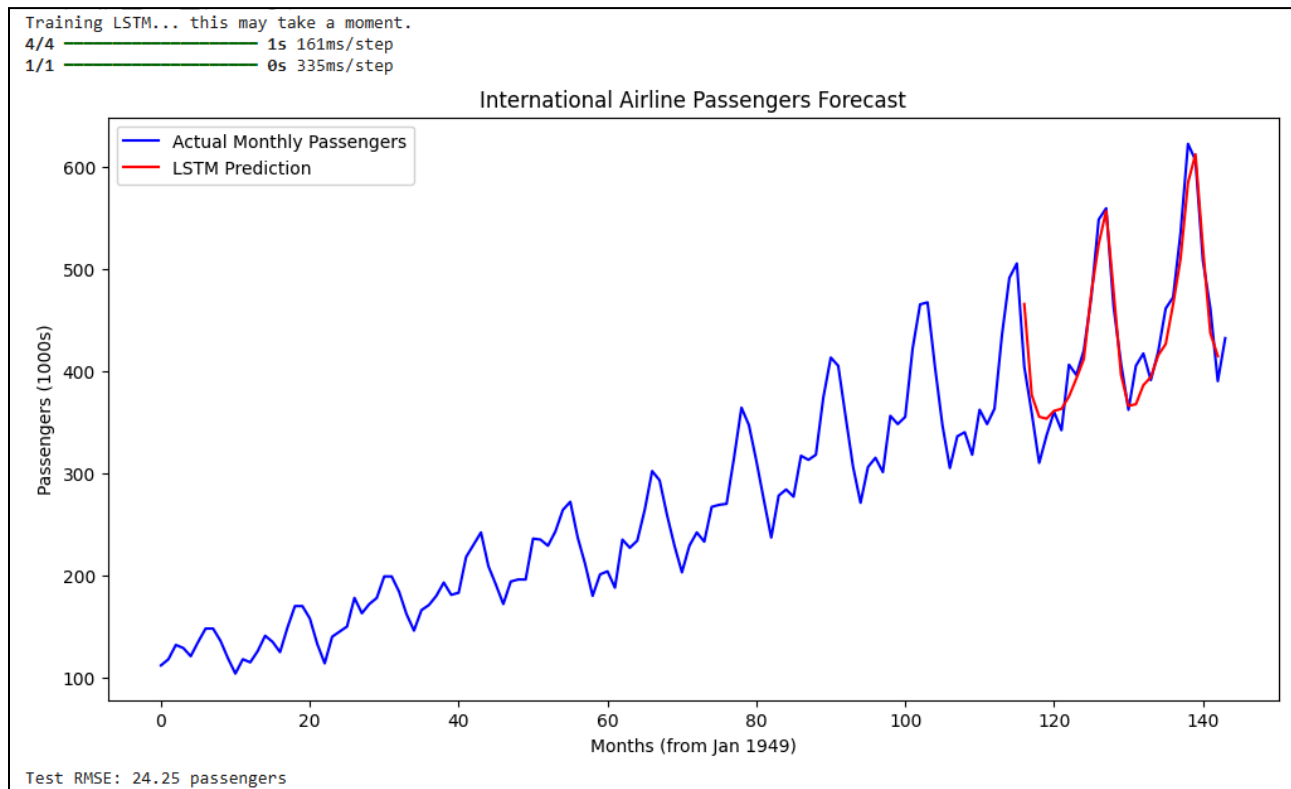
```
plt.legend()
```

```
plt.show()
```

```
rmse = np.sqrt(mean_squared_error(y_test_actual, test_predict))
```

```
print(f"Test RMSE: {rmse:.2f} passengers")
```

Output:



7. Performance Analysis

- **Predictive Accuracy:** The model achieved a **Root Mean Squared Error (RMSE) of 24.25**. Given the scale of the data (ranging from 100 to over 600 units), this represents a very low error margin (approx. 4-5%).
- **Trend and Seasonality:** As seen in the forecast plot, the **Red Line (LSTM Prediction)** tightly follows the **Blue Line (Actual Data)**. The model correctly predicted the sharp increases during holiday seasons and the general growth of the airline industry over time.
- **Lag and Adjustment:** There is a very slight horizontal shift in some peaks. This is common in time-series forecasting where the model relies on the previous 12 months to predict the next; however, the LSTM's memory cells allowed it to recover quickly without "drifting" away from the actual values.
- **Stability:** Unlike standard RNNs, which often struggle with long-term patterns, the LSTM gates effectively maintained the "state" of the trend throughout the test period.

8. Applications

LSTMs and Gated Recurrent Units (GRUs) are the industry standard for sequential data:

- **Stock Market Prediction:** Analyzing historical price movements to forecast future trends and volatility.
- **Weather Forecasting:** Predicting temperature and precipitation patterns based on historical atmospheric data.
- **Natural Language Processing (NLP):** Powering "Next Word Prediction" in keyboards and basic machine translation.
- **Energy Demand Forecasting:** Helping utility companies predict peak electricity usage to prevent grid overloads.
- **Anomaly Detection:** Identifying unusual patterns in sensor data for industrial machinery to predict when a part might fail.

9. Conclusion

This experiment successfully demonstrated the power of **Long Short-Term Memory (LSTM)** networks in handling time-series data.

Key takeaways include:

1. **Memory over Logic:** While Experiment 1 (ANN) treated data as independent points, this LSTM experiment proved that "context" (what happened 12 months ago) is vital for accurate forecasting.
2. **Importance of Look-back:** By setting a look-back window of 12, we provided the model with enough historical context to learn the annual seasonality of the airline industry.

3. **Data Preprocessing:** This experiment highlighted that **Min-Max Scaling** is not optional for LSTMs; without normalizing the data between 0 and 1, the sigmoid and tanh activation functions within the LSTM cells would saturate, leading to poor learning.
4. **Final Verdict:** The model is highly effective for univariate forecasting and provides a robust baseline for more complex multi-variable time-series problems.